Les TRIGGERS

Délio Duchesne-Brajon
Groupe 1

Sommaire

Exercice 1 : virement entre comptes 2

Test 3

Exercice 1 – partie 2 4

Test 5

Exercice 2 : création d'un nouveau client 6

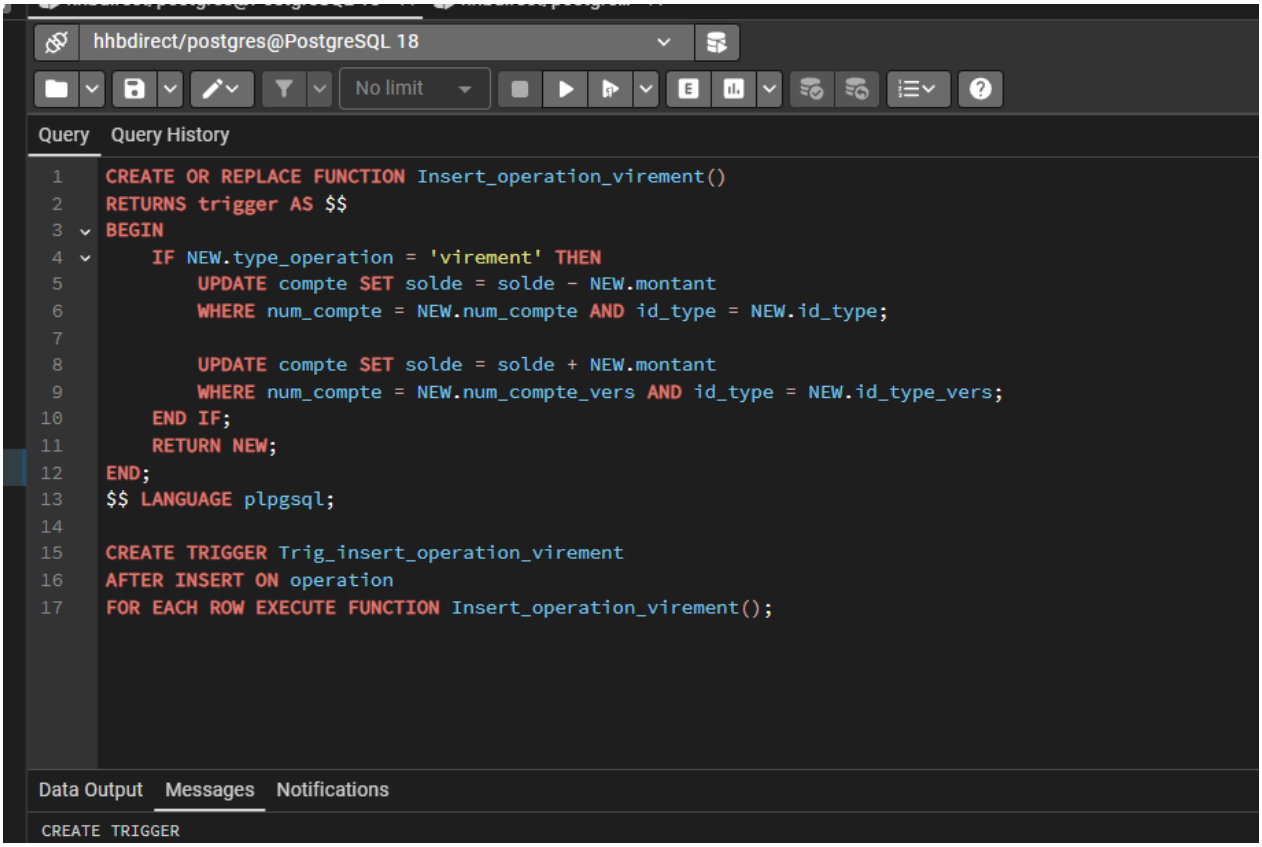
Test 7

Exercice 3 : contrôle des périodes de rémunération 8

Test 9

Exercice 1 : virement entre comptes

Création de la fonction *insert_operation_virement()* et du trigger *Trig_insert_operation_virement*, déclenché après chaque insertion dans la table *operation*. Lorsque l'opération est un virement, le solde du compte débité et celui du compte crédité sont mis à jour.



Test

```
        WHERE num_compte = NEW.num_compte_vers AND id_type = NEW.id_type_
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Trig_insert_operation_virement
AFTER INSERT ON operation
FOR EACH ROW EXECUTE FUNCTION Insert_operation_virement();
 SELECT virement(1001, 1002, 1, 1, 300, 'Loyer');
```

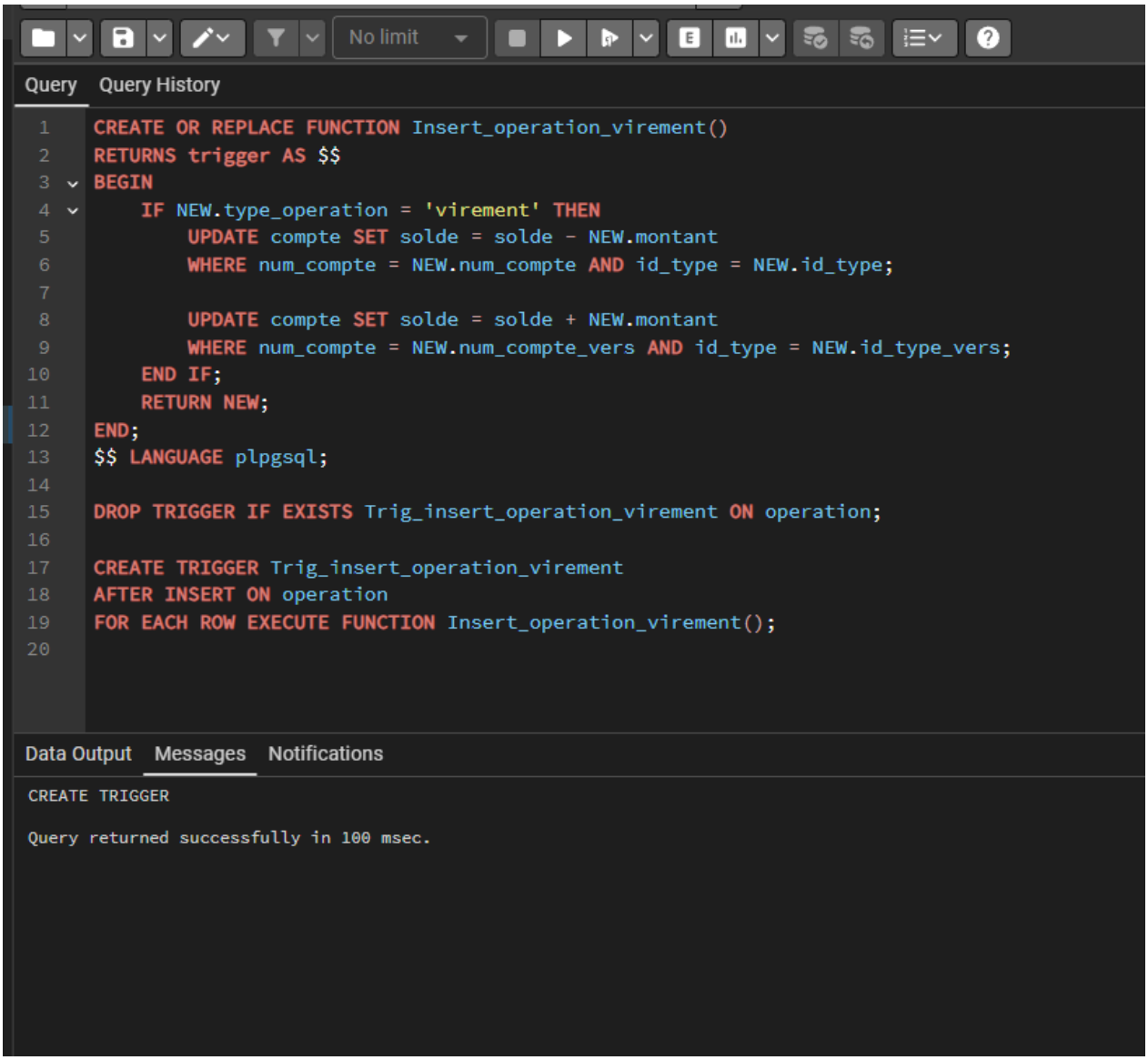
Data Output | Messages | Notifications

SQL

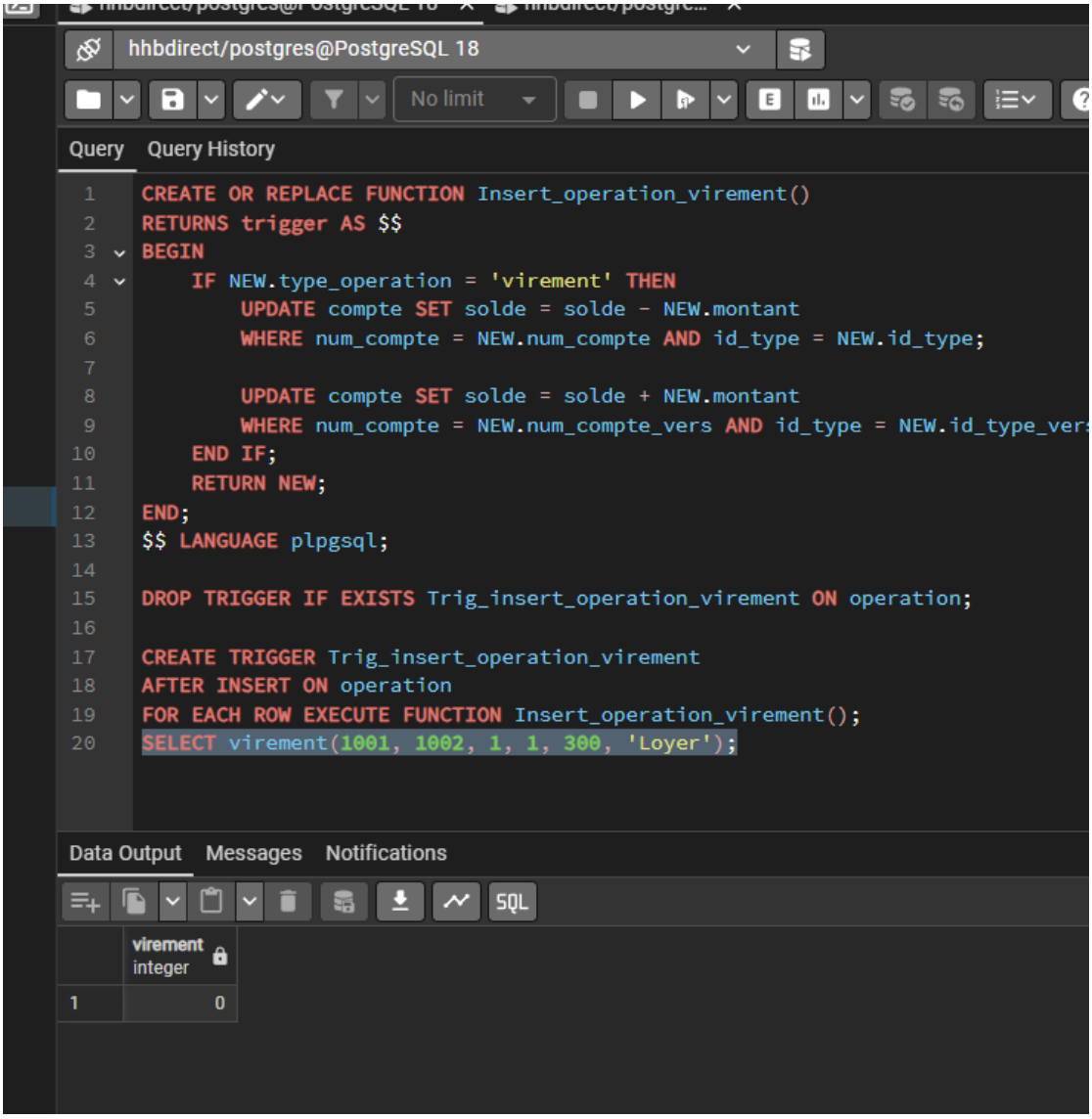
	virement integer
1	0

Exercice 1 – partie 2

Modification de la fonction pour prendre en compte le type du compte destinataire (*id_type_vers*). L'ancien trigger est supprimé puis recréé.

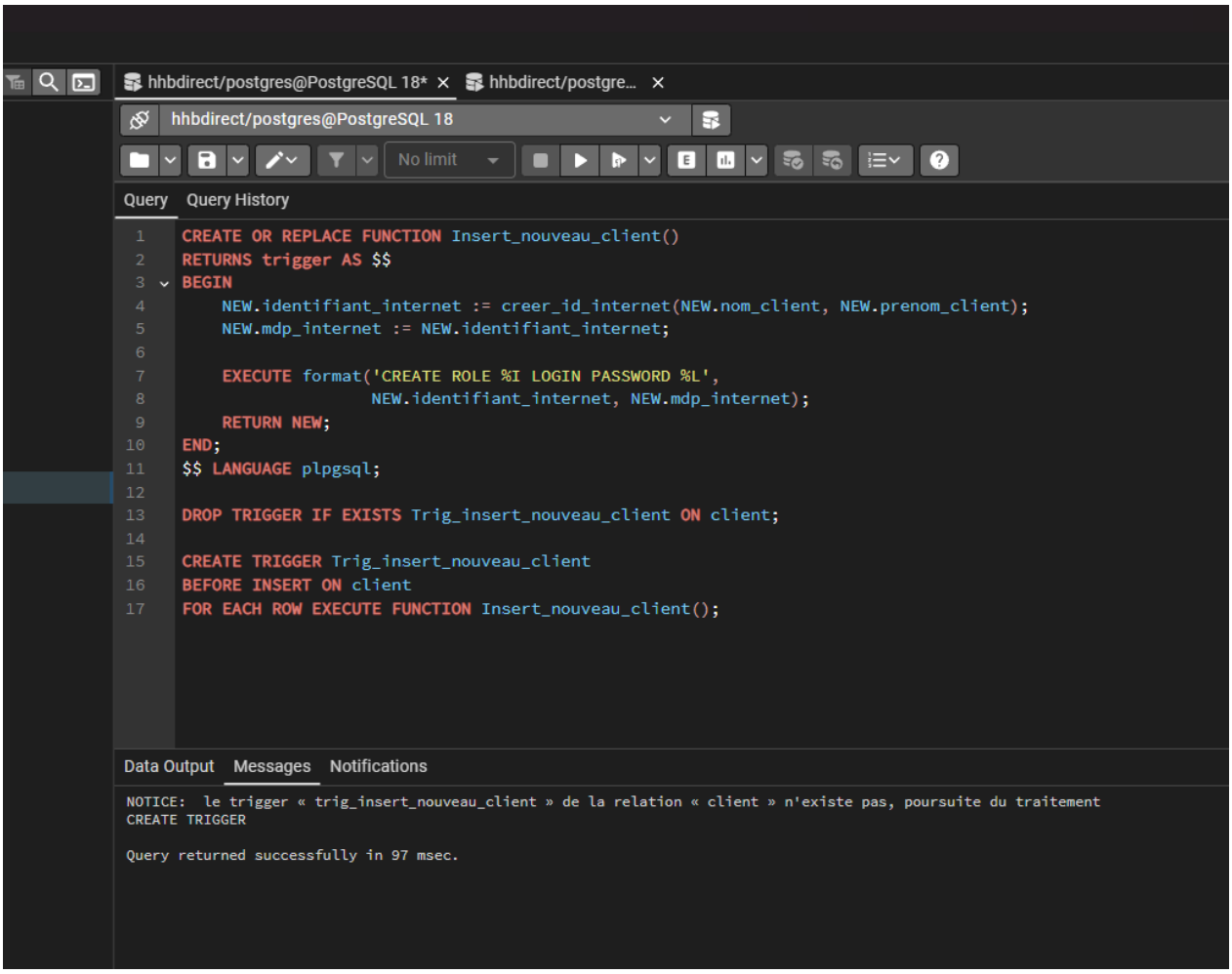


Test


hhbdirect/postgres@PostgreSQL 18
No limit
Query
Query History
CREATE OR REPLACE FUNCTION Insert_operation_virement()
RETURNS trigger AS $$
BEGIN
IF NEW.type_operation = 'virement' THEN
UPDATE compte SET solde = solde - NEW.montant
WHERE num_compte = NEW.num_compte AND id_type = NEW.id_type;
UPDATE compte SET solde = solde + NEW.montant
WHERE num_compte = NEW.num_compte_vers AND id_type = NEW.id_type_vers
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
DROP TRIGGER IF EXISTS Trig_insert_operation_virement ON operation;
CREATE TRIGGER Trig_insert_operation_virement
AFTER INSERT ON operation
FOR EACH ROW EXECUTE FUNCTION Insert_operation_virement();
SELECT virement(1001, 1002, 1, 1, 300, 'Loyer');
Data Output
Messages
Notifications
SQL
virement
integer
1
0

Exercice 2 : création d’un nouveau client

Le trigger *Trig_insert_nouveau_client* s'exécute avant l'insertion d'un client. Il génère l'identifiant internet et le mot de passe, puis crée le rôle PostgreSQL correspondant.



Test

```
CREATE TRIGGER Trig_insert_nouveau_client
BEFORE INSERT ON client
FOR EACH ROW EXECUTE FUNCTION Insert_nouveau_client();
INSERT INTO client (num_client, nom_client, prenom_client, adresse_client)
VALUES ((SELECT MAX(num_client) + 1 FROM client), 'Dupont', 'Marie', '5 rue Pasteur, Manosque');
```

Data Output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 77 msec.
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS Trig_insert_nouveau_client ON client;

CREATE TRIGGER Trig_insert_nouveau_client
BEFORE INSERT ON client
FOR EACH ROW EXECUTE FUNCTION Insert_nouveau_client();
SELECT * FROM client ORDER BY num_client DESC LIMIT 1;
```

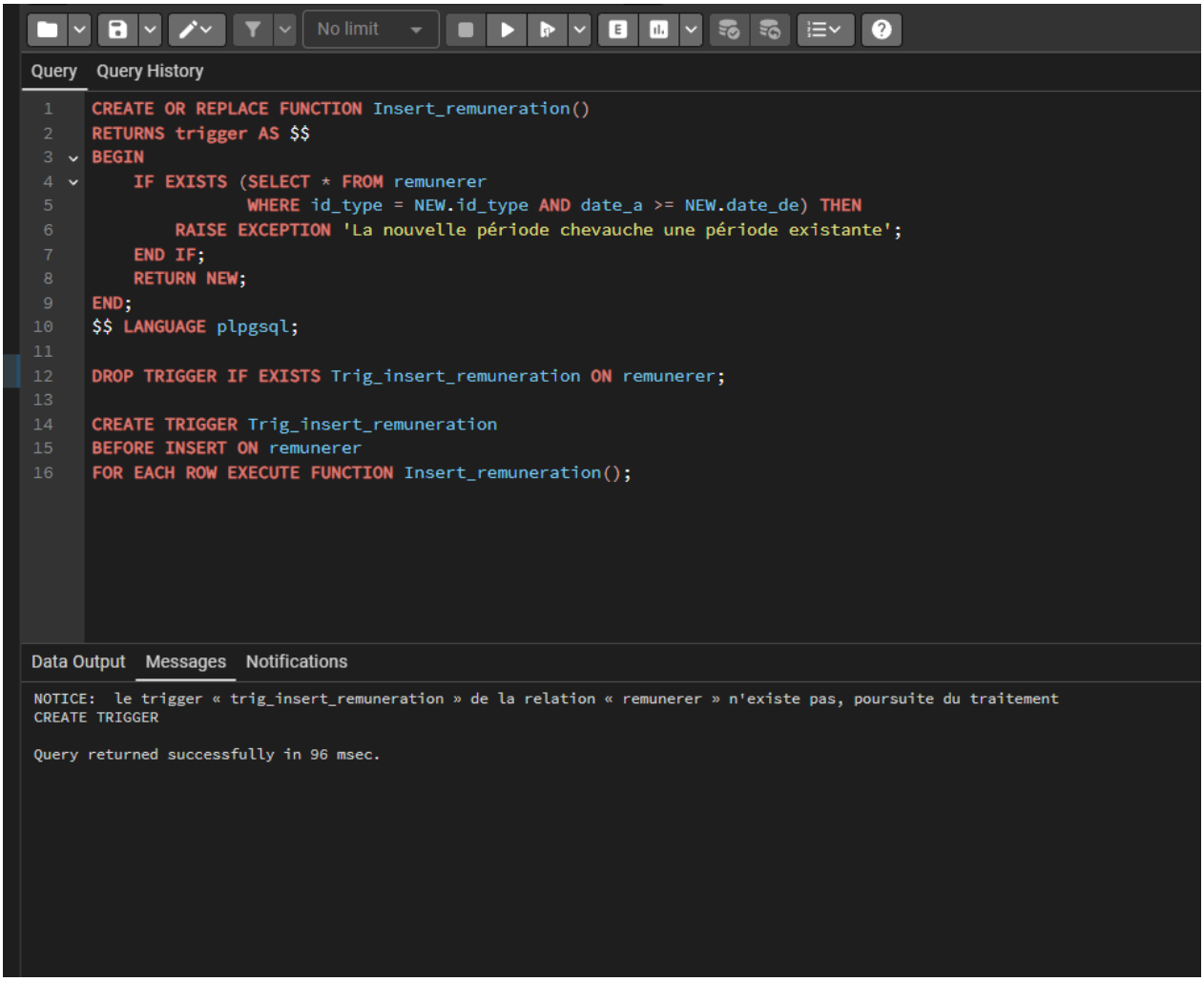
Data Output | Messages | Notifications

SQL — Showing rows: 1

	num_client [PK] integer	nom_client character varying (30)	prenom_client character varying (30)	adresse_client character varying (192)	identifiant_internet character varying (30)	mdp_internet character varying (30)
1	9	Dupont	Marie	5 rue Pasteur, Manosque	Mdupont2	Mdupont2

Exercice 3 : contrôle des périodes de rémunération

Le trigger *Trig_insert_remuneration* s'exécute avant l'insertion dans la table *remunerer*. Si la nouvelle période chevauche une période existante pour le même type de compte, une exception est levée et l'insertion est refusée.



Test

Query | Query History

```
1  INSERT INTO remunerer (id_type, date_de, date, taux_interet)
2  SELECT id_type, date_de, date, 1 FROM remunerer LIMIT 1;
```

Data Output | Messages | Notifications

```
ERROR:  La nouvelle période chevauche une période existante
CONTEXT:  PL/pgSQL function insert_remuneration() line 6 at RAISE

ERREUR:  La nouvelle période chevauche une période existante
SQL state: P0001
```